

SmartZone: Runtime Support for Secure and Efficient On-Device Inference on ARM TrustZone

Zhaolong Jian , Xu Liu , Qiankun Dong , Longkai Cheng , Xueshuo Xie ,
and Tao Li , *Member, IEEE*

Abstract—On-device inference is a burgeoning paradigm that performs model inference locally on end devices, allowing private data to remain local. ARM TrustZone as a widely supported trusted execution environment has been applied to provide confidentiality protection for on-device inference. However, with the rise of large-scale models like large language models (LLMs), TrustZone-based on-device inference faces challenges in migration difficulties and inefficient execution. The rudimentary TEE OS on TrustZone lacks both the inference runtime needed for building models and the parallel support necessary to accelerate inference. Moreover, the limited secure memory resources on end devices further constrain the model size and degrade performance. In this paper, we propose SmartZone to provide runtime support for secure and efficient on-device inference on TrustZone. SmartZone consists three main components: (1) a trusted inference-oriented operator set, providing the underlying mechanisms adapted to the TrustZone’s execution mode for trusted inference of DNN models and LLMs. (2) the proactive multi-threading parallel support, which increases the number of CPU cores in the secure state via cross-world thread collaboration to achieve parallelism, and (3) the on-demand secure memory management method, which statically allocates the appropriate secure memory size based on pre-execution resource analysis. We implement a prototype of SmartZone on the Raspberry Pi 3B+ board and evaluate it on four well-known DNN models and llama2 LLM. Extensive experimental results show that SmartZone provides end-to-end protection for on-device inference

while maintaining excellent performance. Compared to the origin trusted inference, SmartZone accelerates the inference speed by up to $4.26\times$ and reduces energy consumption by 65.81%.

Index Terms—Trusted on-device inference, ARM TrustZone, parallel support, confidential computing.

I. INTRODUCTION

RECENTLY, end devices have served as crucial platforms for deploying deep learning models [1]. Various model inference applications have been deployed on end devices such as embedded devices and mobile devices, unleashing their values in diverse scenarios. For instance, on embedded devices such as fundus cameras, medical image recognition models have been deployed to assist doctors in diagnosing diseases [2]. On mobile devices like smartphones, large language models (LLMs) have been integrated to function as intelligent chat assistants [3]. This paradigm of on-device inference conducts computations locally, thereby avoiding data privacy breaches and latency overhead when sending data to cloud servers. However, in the absence of additional measures for confidentiality preservation, malicious OS or privileged software can easily compromise the execution of on-device inference. Such compromise can result in the leakage of sensitive patient or user privacy, leading to severe consequences [4].

Fortunately, ARM TrustZone provides a viable solution for achieving trusted on-device inference. Since ARM processors have already dominated the IoT and mobile platforms [5], TrustZone has garnered widespread across these devices as a trusted execution environment (TEE) solution. TrustZone isolates specific software and hardware resources to create a protected enclave within the processor. This isolation mechanism shields programs and data in the enclave from various external threats, including attacks from other applications, privileged software, and hardware. Consequently, TrustZone enables the preservation of the confidentiality and integrity of applications running on untrusted platforms.

Although TrustZone provides a basis for trusted on-device inference, its integration faces several challenges. Firstly, TrustZone lacks runtime support for model inference, requiring manual implementation of complex operators and application rewriting, a labor-intensive process as demonstrated in previous work [4], [6]. While migrating existing deep learning

Received 14 November 2024; revised 23 March 2025; accepted 28 March 2025. Date of publication 8 April 2025; date of current version 12 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62272248; in part by the Natural Science Foundation of Tianjin under Grant 23JCZDJC01010 and Grant 23JCQNJC00010; in part by the Key Program of Science and Technology Foundation of Tianjin under Grant 24HHXCSS00004; and in part by the Tianjin Graduate Scientific Research Innovation Project under Grant 2022BKY028. Recommended for acceptance by X. Fu. (*Corresponding authors: Xueshuo Xie; Tao Li.*)

Zhaolong Jian, Xu Liu, Qiankun Dong, and Longkai Cheng are with the College of Computer Science, Nankai University, Tianjin 300350, China, and also with Tianjin Key Laboratory of Network and Data Science Technology, Tianjin 300350, China (e-mail: jianzhaolong@mail.nankai.edu.cn; liu.x@mail.nankai.edu.cn; qiankund@nankai.edu.cn; chenglk@mail.nankai.edu.cn).

Xueshuo Xie is with Haihe Lab of ITAI, Tianjin 300450, China (e-mail: xueshuoxie@nankai.edu.cn).

Tao Li is with the College of Computer Science, Nankai University, Tianjin 300350, China, and also with Haihe Lab of ITAI, Tianjin 300450, China (e-mail: litao@nankai.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TC.2025.3557971>, provided by the authors.

Digital Object Identifier 10.1109/TC.2025.3557971

frameworks can simplify this transition, their large codebases significantly enlarge the attack surface, violating the principle of minimizing the trusted computing base (TCB). Moreover, the limited underlying dependent libraries supported by TEE OS complicate application migration [7], [8]. Secondly, the most commonly used TEE OS, OP-TEE, lacks parallel support [9]. Hence, compute-intensive model inferences with good parallelism perform inefficiently under TrustZone's single-thread execution mode [9], and cannot achieve performance improvements using common multi-threading methods. Thirdly, the limited secure resources available on TrustZone conflict with the growing memory requirements of model inference. Previous research has addressed memory limitations through model partitioning, which leads to either inadequate protection or significant performance degradation [6], [10].

In this paper, we propose SmartZone, the runtime support for on-device inference on TrustZone, ensuring full process protection while keeping efficient execution. SmartZone extends our previous work [11], which adds support for LLM inference and introduces multi-threaded parallelism to improve execution performance. To this end, SmartZone first designs a trusted inference-oriented operator set adapted to TrustZone's execution mechanism for DNN models and LLMs. Leveraging operator selection and combination, SmartZone is capable of constructing various typical models, thereby offering compatible and extensible runtime support. Furthermore, SmartZone implements the proactive multi-threading support on the TEE side through cross-world thread collaboration. This approach creates a twin thread on the rich execution environment (REE) side paired with each secure thread and handles the secure thread control flow from trusted application (TA) in the REE daemon thread through extended system calls. The CPU cores hosting the twin threads then switch to the secure state, taking on computing tasks to markedly speed up the inference process. Lastly, SmartZone designs the on-demand secure memory expansion method, estimating the memory requirement using pre-execution resource analysis and then statically modifying the memory layout to expand the secure memory. This method helps avoid performance degradation due to model partitioning and multiple executions while minimizing TCB inflation as much as possible. Our contributions can be summarized as follows:

- We design a trusted inference-oriented operator set with hybrid granularity operators, offering underlying runtime support adapted to TrustZone's mechanism. This approach supports the trusted inference of typical DNN models and LLMs, leading to a 29.67% average performance gain via the optimized mathematical operators.
- We propose the proactive multi-threading parallel method that actively increases the number of CPU cores in the secure state through cross-world thread collaboration. This method attains up to a 3.13 $\times$ speedup on the quad-core testbed when processing computation-heavy workloads.
- We present the on-demand secure memory management mechanism, which statically allocates the appropriate secure memory size based on pre-execution resource analysis. This mechanism allows efficient inference of large-scale models with only 0.17% latency overhead.

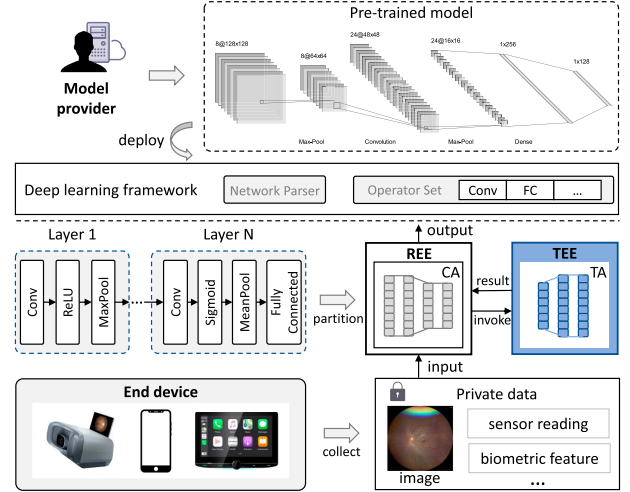


Fig. 1. On-device inference deployment and workflow.

- We implement the SmartZone prototype on the Raspberry Pi 3B+ board and perform the complete inference process on four widely-used DNN models and llama2 LLM. Compared to the original inference on the TEE side, SmartZone significantly improves inference speed by up to 4.26 $\times$ and reduces energy consumption by 65.81%.

II. BACKGROUND & MOTIVATION

In this section, we draw the motivation and challenges in designing SmartZone from threefold. We first introduce the typical scenario and execution process of on-device inference, point out the requirements for trusted execution, and compare it with related work. Then, we delve into the architecture of TrustZone, analyzing the flaws caused by its execution mechanisms in applications. Finally, we define the design goal of SmartZone and summarize the associated challenges.

A. On-Device Inference: Workflow and Security Requirements

On-device deep learning inference. On-device inference represents a pivotal shift in deploying machine learning models, bringing deep learning computation to the data-originating end devices. As shown in Fig. 1, the workflow of on-device inference generally involves several key steps. Initially, the model provider trains a deep-learning model tailored for specific tasks, potentially incorporating their private data. The pre-trained model is then deployed onto the device. The model subsequently performs inference using the collected data such as images or sensor readings, providing inference results instantaneously. During these processes, deep learning frameworks offer vital runtime support through a rich set of pre-defined operators to perform various mathematical and logical functions, such as matrix multiplications, convolutions, activation functions, and pooling layers. These operators are meticulously optimized for high performance across various hardware platforms, leveraging GPU acceleration, multi-threading, and

TABLE I
COMPARISON OF RELATED WORK ON TEE-BASED TRUSTED ON-DEVICE INFERENCE

Remarks: ○: the feature is **not satisfied**; ◐: the feature is **partially satisfied**; ●: the feature is **fully satisfied**

Type	Key idea	Research	Compatibility	Extensibility	Performance			Security
					Compared with REE	Compared with TEE	Precision	
Model Partitioning	Protect entire model	Trusted-DNN [6] T-SLICES [10]	●	○	Slow Slow	/ /	– –	● ●
	Multi-enclave	Branchy-TEE [12] Lasagna [13]	○	○	Comparable Comparable	Fast Fast	– –	● ●
	Protect sensitive layers	DarknetTZ [4] AegisDNN [14]	●	○	Comparable Comparable	Fast Fast	– –	◐ ◐
	Computation offloading	Slalom [15]	○	○	Fast	Fast	–	◐
		Hybrid DL [16]	○	○	Fast	Fast	↓	◐
		HybridTEE [17]	○	○	Slow	Fast	–	◐
		Origami Inference [18]	○	○	Fast	Fast	–	◐
		DarKnight [19]	○	○	Fast	Fast	↓	◐
		ENIGMA [20]	○	○	Fast	Fast	–	◐
		SOTER [21]	○	●	Fast	Fast	–	◐
		ShadowNet [22]	○	○	Fast	Fast	–	◐
Model Optimization	Progressive pruning	Bin Hu et al. [23]	●	○	Slow	Fast	↓	●
	Dual-branch model	TEESlice [24]	●	○	Comparable	Fast	↓	◐
		MirrorNet [25] TBNNet [26]	● ●	○ ○	Comparable Comparable	Fast Fast	↓ ↓	◐ ◐
TEE Optimization	Heterogeneous TEE	SecDeep [27]	○	●	Fast	Fast	–	●
		GuardNN [28]	○	●	Fast	Fast	–	●
		CROUNS [29]	○	●	Fast	Fast	–	●
		StrongBox [30]	○	●	Fast	Fast	–	●
	Confidential VM	GuaranTEE [31]	○	●	Comparable	Fast	–	●

other performance-enhancing techniques. Additionally, users can also define custom operators to extend the framework's functionality when the pre-defined ones do not meet their specific needs. By abstracting hardware architectures and the low-level computation details, the runtime allows users to execute their models on various devices without modification.

TEE-based trusted on-device inference. On-device inference paradigm eliminates the high latency and data privacy leakage risks during the offloading of computations to the cloud. However, the private models deployed on end devices, along with the user data stored on the devices, still face various security threats from malicious OS or privileged software. Previous research has proven that the isolated environment provided by TEE can effectively safeguard the on-device inference process. Table I summarizes and compares this research, which can be categorized into three types:

1) *Model Partitioning*: The first type of research focuses on adapting to the TEE's execution mode through model partitioning. The naive idea is to partition the model to match the resource size of TEE and then load each part individually into TEE for computation [6], [10]. This method can be directly implemented on existing TEEs, providing full protection. However, it lacks extensibility and suffers from significant performance degradation due to the multiple world switches. To alleviate resource constraints, some research utilizes multiple enclaves or TEEs [12], [13]. This solution, however, is only suitable for setups with TEE clusters. More research chooses to protect only a subset of the model via model partition, determining to protect the shallow [16], [18], deep [4], intermediate [20], [21], or nonlinear layers [15], [22] based on different security assumptions. The remaining parts of the model are

typically offloaded to the REE [4], [14], [16], cloud TEE with more resources [17], or heterogeneous accelerators for execution [4], [15], [18], [19], [20], [21]. While these studies sacrifice complete protection in exchange for improved inference performance, they also incur communication overhead during computation offloading. Some of them incorporate additional security enhancements in the REE, such as data encryption or obfuscation, which may increase latency and potentially reduce inference precision [16], [19].

2) *Model Optimization*: The second type of research is dedicated to optimizing the model itself to better adapt to the limited resources available in TEE. For instance, Bin Hu et. al. aims to reduce the resource demands of DNN model inference through progressive pruning and storage optimization [23]. These efforts help in reducing the secure memory footprint required to run complex models within the constrained environment of TEE. Some research [24], [25], [26] have designed dual-branch model structures. This approach places the non-sensitive main network in the REE and the lightweight model branch containing sensitive weights in the TEE. The full function of the model is realized only through the collaboration of the two branches. These methods for reducing model size, including traditional model quantization and pruning, are orthogonal to the security mechanisms provided by TEE, meaning they can be directly applied with model partitioning methods. However, these optimization methods require redesigning the models and may impact the model accuracy to varying degrees.

3) *TEE Optimization*: The third type of research is centered around augmenting TEE architecture to accommodate more secure resources and a richer runtime environment. Some research aims to construct heterogeneous TEE, enabling hardware-level

isolation on accelerators like GPU, NPU, and TPU [27], [28], [29], [30]. Leveraging these heterogeneous TEEs can greatly improve the performance of on-device inference while ensuring security. Indeed, this approach necessitates incorporating specific accelerator drivers to provide runtime support. The large TCB and strong hardware dependencies make this approach difficult to apply to existing devices. There is also research that performs model inference on advanced TEE hardware that provides VM-level isolation [31]. This method also provides comprehensive runtime support and has extensibility. However, it also relies on special hardware, such as the ARM CCA. Devices that support CCA are still several years away from commercial use. At the same time, updating or replacing existing devices with those that support more advanced TEE architectures will incur significant economic costs.

The comparison reveals that current research hasn't yet discovered a method that fully addresses compatibility, extensibility, performance, and security. The TEE optimization approach appears the most comprehensive but faces barriers to being applied on most end devices due to specialized hardware requirements and the large TCB brought by the complete runtime. Given these constraints, the most practical approach for most existing end devices is to enhance scalability and performance by adding runtime support to the TEE OS while ensuring complete protection. This software-level optimization strikes a balance between enhanced security and performance, without requiring extensive hardware modifications, making it more feasible for widespread adoption on existing end devices.

B. ARM TrustZone: Architecture and Execution Mechanism

TrustZone architecture. As shown in Fig. 2, TrustZone establishes two separate virtual environments: the secure world (TEE) and the normal world (REE). The normal world runs the standard OS such as Linux and ordinary applications without protection, while the secure world runs the TEE OS and TAs, completely isolated from REE. As for the software architecture, TrustZone consists of four exception levels, EL0-EL3, with ascending privileges, hosting applications, operating systems, hypervisor/secure partition manager (SPM, introduced in ARM v8.4), and low-level firmware—secure monitor, respectively. The switch between the two worlds is triggered by a special interrupt called Secure Monitor Call (SMC), processed by the secure monitor at EL3. This switching process is notably time-consuming, taking nearly 6,000 clock cycles [7]. At the hardware level, TrustZone uses the *SCR_EL3.NS* flag bit of the *SCR_EL3* register to indicate the secure state, enforcing isolation of memory, peripherals, and interrupts through secure state checks. Hardware components like the TrustZone address space controller (TZASC) and TrustZone memory adapter (TZMA) ensure memory isolation by partitioning DRAM, on-chip ROM, and SRAM into different secure or non-secure regions. Likewise, the TrustZone protection controller (TZPC) provides peripheral isolation by configuring the secure state of peripherals. These mechanisms collectively ensure that TA's code and data are securely loaded into protected memory areas, with access strictly limited to CPUs in the secure state.

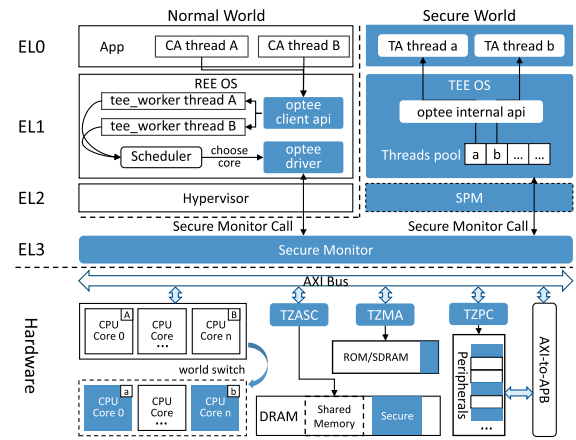


Fig. 2. Architecture and execution mechanism on ARM TrustZone.

TrustZone execution mechanism. The execution of TAs relies on the interfaces and runtime provided by the TEE OS, with OP-TEE being the most widely used [30], [32], [33]. Key components of OP-TEE include the OP-TEE client API, tee-supplciant and OP-TEE driver on the REE side, and the OP-TEE internal API on the TEE side. In a typical TA execution process, users first invoke the TA by executing the corresponding client application (CA) on the REE side via OP-TEE client APIs. The tee-supplciant process establishes an RPC session for this invocation, which is then handled by the OP-TEE driver within the Linux kernel, triggering an SMC interrupt. The secure monitor processes this interrupt, switching the CPU to the secure state. Next, OP-TEE creates a thread to execute TA's predefined functions. Upon completion, the execution result is returned back to the CA through reverse workflow. Data transfer between the CA and TA is achieved through shared memory, with data being encrypted and decrypted at each end for secure processing.

C. SmartZone: Challenges and Goals

Analyzing the potential flaws of the TrustZone execution mechanism reveals several challenges for trusted on-device inference. The execution of TAs is constrained by the rudimentary OP-TEE OS, which provides only limited libraries and lacks support for model inference. Academic efforts to address this issue have mainly focused on retaining a subset of the model on the TEE side, without providing generalized runtime support like operators [23], [27]. Moreover, TAs are executed in a single-thread mode since OP-TEE lacks the multi-threading capability, which is a widely recognized concern in the community¹. Parallelism in OP-TEE is achieved only through the concurrent TAs, requiring users to partition their applications into multiple TAs and implement inter-TA communication via shared memory. This method increases the user's burden and degrades performance due to the encrypted communication overhead. Additionally, the limited secure memory is insufficient for on-device inference. For instance, the Raspberry Pi

¹OPTEE document, frequently asked questions (<https://optee.readthedocs.io/en/latest/faq/faq.html#q-is-multi-threading-supported-in-a-ta>).

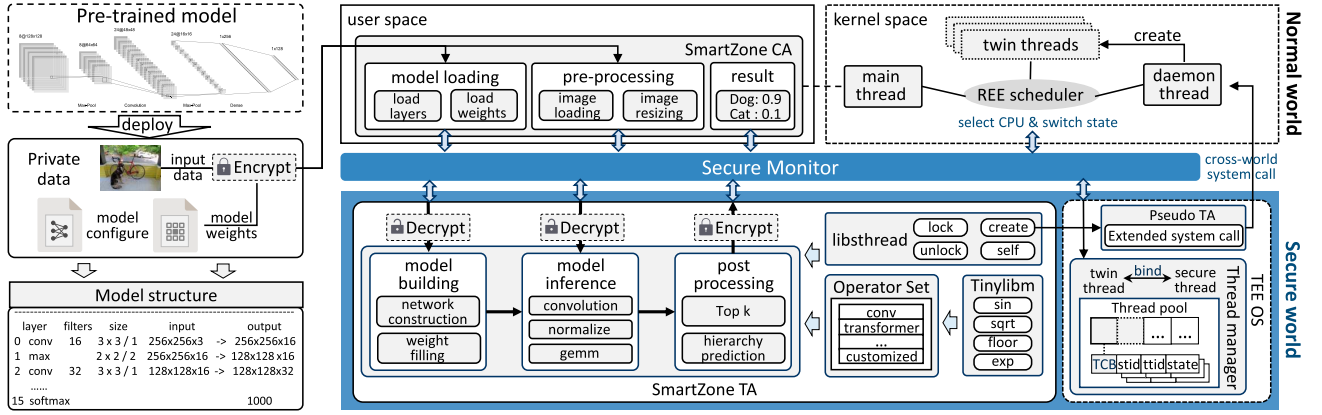


Fig. 3. SmartZone architecture overview.

3B+ board has only 16 MB of secure memory, which is even smaller than the weight size of most models [4], [11]. Partitioning the model to protect only the critical subset reduces security while complete protection through multiple executions slows down execution. These limitations hinder the effectiveness of TrustZone-based trusted on-device inference.

To overcome these threefold challenges, we propose SmartZone to provide runtime for trusted on-device inference with optimization mechanisms to achieve parallel processing and secure memory expansion. SmartZone can be seen as a complement to the OP-TEE runtime, which aims to provide complete protection throughout the entire on-device inference process while achieving efficient inference. The design goals of SmartZone include:

- **Compatibility.** SmartZone crafts a universal runtime tailored to the on-device inference, and can seamlessly integrate on common TrustZone-enabled end devices.
- **Extensibility.** SmartZone sets up an operator set to support DNN and LLM model inferences at the base level, and can adapt to more models through operator extension.
- **Performance.** SmartZone boosts performance of trusted on-device inference by introducing multi-threading parallelism to OP-TEE via cross-world thread collaboration.
- **Security.** SmartZone enhances secure memory management to provide sufficient memory for models with high memory requirements, thus ensuring complete protection.

III. THREAT MODEL

SmartZone is designed for TrustZone-equipped end devices, aiming to provide comprehensive protection for on-device inference. SmartZone adheres to the common threat model for trusted inference [4], [11], assuming that the hardware implementation and software components (including the bootloader, firmware, secure monitor, and OP-TEE OS) of TrustZone are trustworthy. These components are capable of preserving the integrity and confidentiality of the data and computations in the secure world. In contrast, the user space and REE OS in the normal world are considered untrusted. Adversaries may compromise the REE OS, thereby invading, forging, and modifying inference tasks to steal model weights and user data.

The pre-trained models used for on-device inference may be trained using proprietary data from servers or other stakeholders. We assume that the model provider can safely deploy the model without any privacy issues, for example, using the existing protection method [34].

Specifically, SmartZone protects the confidentiality and integrity of the inference process, starting from loading model weights and inputs into the secure world, and ending until obtaining the inference result from the secure world. Potential attack vectors within the scope of our work include: (1) **Abusing the models.** The valuable pre-trained model may be stolen for unauthorized use [35]. (2) **Tampering with the models.** Attackers may tamper with the model to jeopardize its integrity, or they may cause the model to incorrectly classify predefined inputs, such as through Trojan attacks [36] and bit-flip attacks [37]. (3) **Thwarting inference process.** An adversary can mislead and intentionally thwart the DNN model inference process by adding noise or injecting fake data to the intermediate results of the model, e.g. fault injection attack [38]. Hardware attacks targeting TrustZone itself, such as hardware Trojan insertion [39], and side-channel attacks [40], are out of the scope of our work. These attacks can be mitigated by existing solutions [41] orthogonal to SmartZone.

IV. BASIC DESIGN

In this section, we present the basic design of SmartZone, including its overall architecture, key components, and execution process. We first detail the design of the trusted inference-oriented operation set, which enables flexible and extensible on-device inference. Then, we describe the proactive multi-threading parallel support, the critical feature that significantly enhances SmartZone's execution performance. Lastly, we propose the on-demand secure memory management mechanism to optimize memory utilization efficiency. These design elements ensure that SmartZone protects the entire inference process while maintaining high performance.

A. SmartZone Overview

As shown in Fig. 3, SmartZone comprises three key components: the SmartZone CA on the REE side, the SmartZone

TA on the TEE side, and runtime support within TEE OS. SmartZone follows the standard process of trusted on-device inference. The model provider deploys the pre-trained model to the end device according to the SmartZone specification. The model is stored in two parts: the model configuration (*cfg*) and the model weights (*wts*). The *cfg* file details the model structure, including layer type, filter, filter size, input size, and output size of each layer. SmartZone can build models efficiently based on the *cfg* file. Both the *cfg* and *wts* are encrypted along with private data on the end device, serving as inputs for executing SmartZone CA.

SmartZone accomplishes trusted on-device inference through the collaboration of CA and TA. Based on the trigger time, the inference process can be divided into several stages within SmartZone CA and TA. Each operation in SmartZone CA is implemented as an independent TA command, which is executed via the world-switching process. Conversely, operations in SmartZone TA are initiated on the TEE side and do not involve the world-switching process. Initially, the model loading module in SmartZone CA loads the *cfg* and *wts* files and parses the model structure. The decrypted layer configurations and weights are then processed by the model building module in SmartZone TA. This module constructs the model layer by layer and fills in the corresponding weights. Following this, the pre-processing module in SmartZone CA loads and adjusts images to appropriate dimensions. These images are then used as inputs for model inference in SmartZone TA, starting the model inference process. During inference, the model performs computations based on the predefined execution logic of each layer operator, such as convolution calculations, normalization, pooling, etc. Afterward, the results are handled by the post-processing module in SmartZone TA, which can, for example, deliver the top K results with the highest probabilities in a multi-classification problem. These results are then encrypted and sent back to SmartZone CA as the final output. This process ensures the confidentiality and integrity of the inference, protecting it from potential attacks.

In the inference process, two modules are pivotal: the operator set and the mathematical computation library *Tinylibm*. The operator set is essential to building models, encompassing fundamental operations like convolution and pooling. These operators are implemented at a mixed granularity and can invoke one another, including layer operators, mathematical calculation operators, and mixed operators. *Tinylibm* supports the underlying computations for model inference and operator calculations, offering functions like square root, exponential, etc. With this design, SmartZone achieves flexible and extensible trusted on-device inference.

SmartZone improves the inference performance by incorporating proactive multi-threading parallelism. Users can manually create secure threads using the provided multi-threading APIs in *libsthread*. These APIs communicate with the daemon thread in the REE OS via extended system calls, ensuring the security of thread control flow. For each secure thread, the daemon thread creates a twin thread on the REE side, which is then scheduled by the REE scheduler to specific CPU cores. The CPU cores hosting twin threads switch to the secure state

TABLE II
TRUSTED INFERENCE-ORIENTED OPERATOR SET WITH HYBRID GRANULARITY OPERATORS IN SMARTZONE

Type	Operator	Key parameters / Description
Control	Crop	(flip, angle, exposure, ...)
	Route	(batch, n, layers, sizes)
	Shortcut	(batch, index, w, h, c, w2, h2, c2)
	Network	(lr, momentum, decay, adam, ...)
DNN	Avgpool	(batch, w, h, c)
	Maxpool	(batch, w, h, c, size, stride, pad)
	Convolution	(groups, activation, normalize, ...)
	Depth_Conv	(groups, activation, normalize, ...)
	Connected	(inputs, outputs, activation, ...)
	Batchnorm	(output, mean, variance, ...)
	Dropout	(inputs, probability, net_num, ...)
	Activations	(type, input)
	Softmax	(temperature, spatial, no_loss, ...)
LLM	Cost	(type, scale, ratio, thresh, ...)
	Tokenizer	Converting text into a sequence of tokens
	Sampler	Determining data samples for inference
Math	Transformer	Base structure of transformer-based model
	Blas	Basic Linear Algebra Subprograms
	Gemm	(matrix1, matrix2)
	Im2col	(image, channels, h, w, ksize, ...)
	Tinylibm	Mathematical computation library

and retrieve a secure thread from the TEE thread pool. SmartZone uses the thread manager to maintain each thread's control block, recording the thread state and binding relationships between threads. Additionally, SmartZone enhances memory efficiency through on-demand secure memory management. It calculates the secure memory required for model inference before execution by analyzing the *cfg* and *wts* files. SmartZone statically allocates the appropriate size of secure memory, thus ensuring the entire inference can be placed on the TEE side. These two designs complement the runtime and ensure the performance and security of SmartZone.

B. Trusted Inference-Oriented Operator Set

Table II presents the trusted inference-oriented operator set of SmartZone, comprising 22 distinct operators. Each operator is dedicatedly designed to perform essential functions essential for trusted DNN and LLM inference. SmartZone can parse the model structure and flexibly combine these operators to construct different models. Since the native OP-TEE adheres to a minimized TCB design, it provides only a C language runtime. The operator set of SmartZone is designed based on the execution mechanism specifications of TrustZone and is implemented in pure C language. The operator set resides at the application layer and can migrate across different TEE platforms with only minor modifications for adaptation.

The operators in SmartZone can be divided into four types: (1) Control operators establish fundamental control flows. The Crop operator adjusts the input dimensions to match the target dimensions, ensuring compatibility with subsequent processes. The Route operator is pivotal for transferring data between layers or modules. The Shortcut operator adds the output from earlier layers to the output of subsequent layers, enabling skip connections in certain network designs. The Network operator constructs the network and orchestrates the inference process. (2) DNN operators handle core layers and computations involved in DNN inference. For example, the Convolution operator performs convolutional calculations, while the Activations

operator provides various activation functions essential for non-linear transformations within neural networks. (3) LLM operators are designed for LLM inference. The Tokenizer operator segments input text into smaller units. The Sampler operator offers various sampling strategies, which are crucial for text sequence generation. The Transformer operator accepts a sequence of tokens to predict subsequent tokens, embodying the core computational mechanism of LLMs. (4) Math operators encompass different levels of underlying mathematical computation support. The Blas and Gemm operator implement basic linear algebra computations and matrix multiplication operations. The Tinylibm is a mathematical computation library designed to fill gaps in mathematical computations in OP-TEE, providing fundamental operations such as logarithmic and square root calculations.

It is noteworthy that the operators in SmartZone vary in granularity and can invoke one another. For example, in the DNN inference process, the Network operator calls multiple DNN operators to construct the model and organizes the computation sequence using the Route operator. DNN operators, in turn, call Math operators for calculations, such as the Convolution operator invoking the Gemm operator. Users can perform complex computations through the combination of operators and multi-level invocations among them. The fusion of operators eliminates redundant calculations and reduces data transmission and memory usage. Moreover, the design of the operators allows for easy extension, enabling users to add custom operators for specific functionalities. This operator-level support enhances the extensibility of SmartZone, making it an adaptable runtime for various models.

C. Proactive Multi-Threading Parallel Support

Parallelism is a foundational optimization technique in model inference, greatly enhancing the inference performance. As described in Section II-B, due to design considerations, OP-TEE lacks native thread management, employing a passive scheduling mode where TEE threads are indirectly managed by the REE OS through corresponding REE threads. Each TA starts with a single REE thread bound to a CPU core in the secure state and executes computational tasks serially according to the REE scheduler's strategy. However, since each CPU core can switch to the secure world independently, TAs can be executed concurrently to statically create multiple threads to achieve parallelism. This parallelism mode requires encrypted data transfer via shared memory due to the memory isolation between TAs, leading to significant encryption/decryption overhead. It may also necessitate TA boundary checks, complicating thread management. Furthermore, concurrent execution requires manual application partitioning, affecting usability. These challenges have resulted in a limited focus on improving the performance of trusted inference through parallel optimization in prior work [6], [10].

Cross-world thread collaboration. SmartZone's proactive multi-threading parallel support involves the collaboration between TEE threads and REE threads. This approach, inspired by TEEp [9], enables the proactive increase in the number

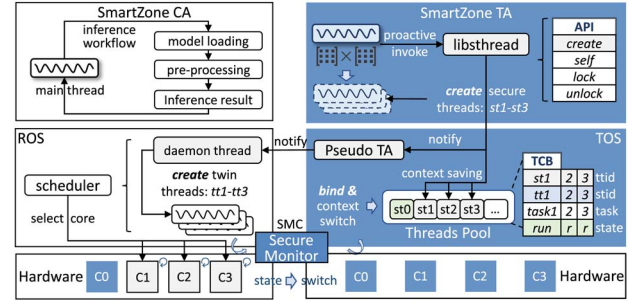


Fig. 4. Proactive multi-thread parallel support.

of CPU cores in the secure state, distinguishing it from the native passive scheduling mode. As shown in Fig. 4, upon initiation of the CA, SmartZone activates a daemon thread in the REE OS that continuously monitors secure thread control flow operations from the TEE side during the execution of the TA. When the TA issues a *libstthread* API call, SmartZone sends this request to the daemon thread via the corresponding system call. These extended cross-world system calls are uniformly handled by the Pseudo TA², which operates at the same secure level as TEE OS. Upon receiving the request to create a secure thread, the daemon thread creates a new kernel thread on the REE side, termed the twin thread. The REE scheduler then assigns the twin thread to an idle CPU core, which switches to the secure state, thereby increasing the number of CPU cores in the secure state and enabling parallel acceleration.

SmartZone designs a thread pool and thread manager in TEE OS to handle thread allocation and binding. The thread pool maintains the control blocks for the main TA thread and all the created secure threads. Each control block records important information such as the secure thread ID, the corresponding twin thread ID, and the thread state. When the CPU core hosting the twin thread switches to the secure state, the thread manager assigns a secure thread from the thread pool to the core. The thread manager then records the twin thread ID into the control block, completing the binding of the cooperating relationship. Unlike TEEp, SmartZone manages threads on both sides in kernel mode rather than user mode to reduce the overhead of privilege level switching. This runtime mechanism enables minimal thread management, thereby supporting multi-threading parallelism on TrustZone.

Proactive parallel programming semantics. To implement proactive multi-threading parallel support, we define the parallel programming semantics necessary for trusted multi-threading support. Drawing on the *pthread* library in Linux, we limit the multi-threading semantics on the TEE side to four basic operations to minimize TCB: thread creation, obtaining thread ID, thread locking, and thread unlocking. Each of these operations corresponds to an API and an extended system call, as shown in Fig. 5. *TEE_sthread_create* is used to create threads in the TA, while *TEE_sthread_self* returns the ID of the current thread for thread management. *TEE_sthread_lock* and

²https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html

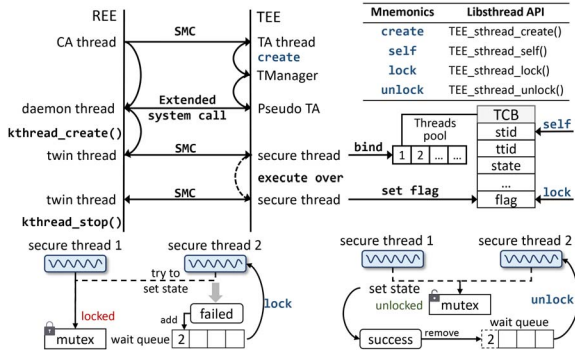


Fig. 5. Proactive parallel programming semantics and workflow.

TEE_sthread_unlock are employed for thread synchronization and, combined with other verification logic, to complete operations related to mutexes and condition variables. This method maintains control over the thread control flow on the TEE side, preventing API misuse by the CA on the REE side, thereby ensuring thread security. With these programming semantics, users can create multi-thread TAs by actively invoking the APIs.

The use of the *libthread* API in SmartZone follows the same workflow of existing multi-threading APIs. Initially, the CA thread adheres to the native execution mechanism of OP-TEE, switching to a secure state through the SMC interrupt to commence the TA thread execution. For parallelizable computation tasks, such as typically matrix multiplication computations, the TA thread creates multiple secure threads using the *TEE_sthread_create* API based on the user-specified parallel mode. SmartZone divides the matrix into several sections according to the number of CPU cores and assigns each section to different threads. The *TEE_sthread_create* API is processed by the Pseudo TA in the TEE OS, with the thread information recorded by the *TManager*. The Pseudo TA communicates with the daemon thread on the REE side via extended system calls, prompting the daemon thread to create a twin thread on the REE side using the *kthread_create* API. The twin thread switches to the secure state through an SMC interrupt, binds with the secure thread, and records the related information in the thread control block. After execution, the secure thread records the results in the control block, returns to the REE side, and terminates the twin thread.

The secure threads created by SmartZone exist within the same TA memory space, eliminating the need for inter-thread data interaction via shared memory. However, this mode can lead to resource sharing among threads, requiring the locking of mutually exclusive resources to ensure correct execution. This problem might occur during parallel execution of inference via model parallelism. In such cases, *TEE_sthread_lock* and *TEE_sthread_unlock* in *libthread* can be used to implement functionality similar to mutex locks. The first secure thread to access the shared resource gets the operating permission. Subsequent threads add their IDs to the waiting queue managed by the *TManager* using the *TEE_sthread_self* API and then be locked via *TEE_sthread_lock*, pausing execution. Once the preceding thread finishes accessing the shared resource, the

TManager wakes up the threads in the waiting queue one by one using *TEE_sthread_unlock*. This process reuses OP-TEE's native thread context save and restore mechanism to ensure that the state of each thread is correctly maintained when resuming, thus maintaining the integrity of parallel workflow. This concept of managing cross-world thread bindings and enforcing trusted control flow can also be applied to other TEE platforms.

D. On-Demand Secure Memory Management

Memory isolation is indeed fundamental to the trustworthy assurance of TrustZone. Unlike other TEEs that perform memory encryption with specialized hardware, TrustZone uses hardware components like TZASC to partition DRAM into several regions, defining base addresses, sizes, and secure attributes for each. TrustZone employs different address translation mechanisms for different secure states to translate virtual addresses to physical addresses, allowing CPU access to secure memory only in secure states. However, TZASC's inability to dynamically configure memory regions means the secure memory size is statically determined before the device boots, typically kept small to minimize the TCB. For instance, the Raspberry Pi 3B+ only has 16 MB of secure memory for TA execution. This limitation restricts the performance of trusted inference on TrustZone. The differences in hardware mechanisms prevent TrustZone from reusing existing secure memory extension methods, thus necessitating dedicated designs. To address this, SmartZone designs the on-demand secure memory management method. It enables larger secure memory by pre-execution resource analysis and on-demand secure memory expansion, allowing for more complex models to run efficiently on the end devices.

Pre-execution Resource Analysis. SmartZone first calculates the required secure memory size before execution. For the on-device inference process, the structure of the pre-trained model is fixed. In SmartZone's workflow, the model configure file *cfg* records detailed model structure information. For DNN models, *cfg* contains the input and output size of each layer, the convolution kernel size of the convolution layer, and so on. Based on this information, we can calculate the runtime memory footprint of the DNN model. For LLMs, *cfg* records the run-state size of the transformer structure. In this checkpoint-based execution mode, we can also calculate the runtime memory footprint of the model. This feature sets a theoretical foundation for SmartZone's pre-execution resource analysis. In addition to the dynamic memory footprint at runtime, the secure memory is also used to load the model weights, i.e., the *wts* file. TEE OS also needs to use secure memory to provide basic system services. The sum of the three is the total required secure memory size.

Predicting secure memory footprint hinges on calculating the runtime memory footprint. For DNN models, static memory includes model weights and inputs and can be calculated by parsing the layer information from the *cfg* file. Operations like convolution and pooling affect the runtime footprint by altering feature map sizes. SmartZone reduces this by freeing secure memory after each layer, with the runtime footprint being the

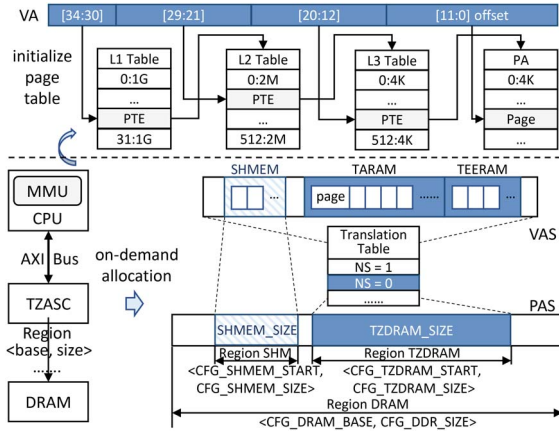


Fig. 6. On-demand secure memory allocation and page table initialization.

maximum of any single layer. For LLMs, the static memory comprises weights and embeddings, while the runtime footprint includes intermediate variables and the key-value caches from attention. These footprints can be calculated by parsing the *cfg* file, with the key-value cache estimated from sequence length, layer dimensions, and attention heads. The total secure memory is the sum of these parts. Pre-execution resource analysis is to estimate a lower bound of required memory rather than the exact value. Practically, setting secure memory slightly above the estimate ensures smooth inference.

On-demand Secure Memory Expansion. After determining the required secure memory size, SmartZone expands the secure memory size on demand through static memory allocation. Fig. 6 shows the detailed process of the on-demand secure memory expansion. Following the memory layout of TrustZone, the secure memory size we refer to is actually the size of TARAM, a memory area in the virtual address space (VAS) obtained through address translation from the TZDRAM domain. Its size is influenced by *TZDRAM_SIZE*, *TEE_RAM_VA_SIZE*, and *SHMEM_SIZE*. *TZDRAM_SIZE* defines the overall secure memory size on the TEE side, with a portion of size *TEE_RAM_VA_SIZE* allocated to TEE OS. *SHMEM_SIZE* specifies the size of shared memory, which does not occupy the secure memory area, but is adjacent to TARAM. An overly large shared memory size can cause memory overlap, leading to privacy leakage risks. Therefore, SmartZone checks the memory boundaries to avoid memory overlap, and then statically sets *TZDRAM_SIZE* based on the required secure memory size to allocate an appropriate TARAM size, thus expanding the secure memory.

For secure memory, OP-TEE uses a three-level page table to map the physical address space to VAS. Each page table corresponds to an MMU page table entry, which can map 2 MB of memory in a 64-bit system. Therefore, SmartZone sets the number of page tables based on the secure memory size to manage the expanded memory. In the native OP-TEE, the page table is initialized each time the TA thread is executed. With small secure memory, the page table remapping overhead is negligible. However, after SmartZone expands the secure memory, this overhead increases. During weight transfer, SmartZone

loads the data in blocks based on the shared memory size. Each load operation triggers the page table remapping, creating a performance bottleneck. To mitigate this, SmartZone only initializes the page table when creating the TA thread for the first time, thus reducing execution latency. When executing in multi-threading mode, SmartZone uses extended system calls for thread management, avoiding triggering page table remapping and ensuring efficient inference.

Tradeoff of secure memory expansion. SmartZone enhances the performance of trusted inference by appropriately expanding the size of secure memory. While this approach does indeed increase the attack surface—for instance, it raises the likelihood of attackers stealing private data from secure memory—we believe that the expansion of secure memory is necessary for several reasons: Firstly, from the perspective of applicability, when secure memory is limited, on-device inference applications need to be completed through model partitioning. The performance overhead induced by multiple executions and state switches is typically several times that of unprotected execution [6], [11]. For real-time tasks on terminal devices, this overhead is unacceptable and significantly impedes the application of trusted inference. Secondly, from the perspective of feasibility, current terminal devices possess sufficient hardware resources to support larger secure memory configurations. For instance, the Raspberry Pi 3B+ board comes equipped with 1 GB of memory, while the maximum secure memory setting we use in the evaluation is a mere 140 MB. Lastly, from the perspective of security, the security risks brought by secure memory expansion can be mitigated by implementing measures such as address space layout randomization (ASLR). Therefore, after carefully balancing performance and security, SmartZone has opted to allocate as small a secure memory as possible while satisfying the requirements of inference applications.

V. EVALUATION

To thoroughly validate the design point and performance benefits of SmartZone, we implement it on the Raspberry Pi 3B+ development board and conduct several evaluations. The evaluation targets include: (1) assessing the overall performance improvements of SmartZone compared to native TrustZone; (2) dissecting the performance gains attributable to each optimization mechanism of SmartZone; and (3) providing insights into how SmartZone outperforms existing research.

A. Experimental Setup

Prototype Implementation. We implement a prototype of SmartZone on Raspberry Pi 3B+ board, which is equipped with the quad-core ARM Cortex-A53 CPU (ARM v8-A 64-bit@1.4GHz) coupled with 1GB of memory. We use Linux-for-arm (version 4.14) as the REE OS to execute the client part of SmartZone and manage other non-secure resources. On the TEE side, we use OP-TEE (version 3.8.0) as the TEE OS to provide trusted runtime. It is worth noting that the Raspberry Pi 3B+ board does not have hardware components such as TZASC and TZPC to implement hardware-level security mechanisms to

TABLE III
THE LOC AND SIZE OF SMARTZONE PROTOTYPE

Componentes	Line of Code	File Size	Compiled Size
SmartZone CA	19,085	812.9KB	573.7KB
SmartZone TA	3,591	201.7KB	168.2KB
Tinylibm	827	65.9KB	40.2KB
SmartZone total	25,289	1,102.3KB	754.1KB
OP-TEE OS	242,849 + 986 ↑	/	/
-thread pool	35	/	/
-TManager	427	/	/
-Pseudo TA	524	/	/
Inflated TCB	5,404		

enforce memory isolation³. After porting Trusted Firmware A (TF-A⁴) and OP-TEE, the Raspberry Pi 3B+ board has acquired complete functional support at the software level, making it a commonly used platform for TrustZone evaluation [6], [7], [11]. This compact prototype helps confirm that SmartZone operates effectively on the end devices with both practicability and extensibility.

As an essential evaluation metric for trusted computing, we thoroughly measure the Lines of Code (LoC) for SmartZone. The results are presented in Table III. Specifically, SmartZone encompasses the CA, the TA, and modifications at the TEE OS level. Among these, the introduction of SmartZone TA and OS-level modifications inflates the TCB. The LoC of SmartZone TA and *Tinylibm* are 3,591 and 827, respectively, and their file sizes are 201.7KB and 65.9KB. After compilation, the file sizes are 168.2KB and 40.2KB, respectively. OS-level modifications primarily include the thread pool and TManager in OP-TEE, as well as the Pseudo TA, which together account for 986 LoC. In total, SmartZone brings an increase of 5404 LoC. Compared to OP-TEE OS with 242849 LoC, the LoC increase of SmartZone is not substantial. By adhering to the design principle of minimizing the TCB, SmartZone achieves support for secure and efficient inference on devices with an acceptable TCB inflate.

Workloads and Metrics. To evaluate the performance of SmartZone, we use four typical DNN models: Mobilenet, Darknet, Resnet18, and Darknet19. The weight file sizes of these models are 17 MB, 28 MB, 44 MB, and 80 MB, respectively. We test the performance of these four DNN models on the ImageNet image classification task. Additionally, to evaluate SmartZone's ability with LLMs, we conduct a fully functional proof of concept using a custom 15M-parameter llama2 model. This scaled-down version features a 288-dimensional space, 6 layers, 6 attention heads, and 6 key-value pair heads, operating with a maximum sequence length of 256 and a 32000-token vocabulary. Trained on the Tinstories dataset [42], the model has 67 MB of weights, adhering to the same training and inference process as llama2.c⁵. This test showcases the practicality of LLM inference within the tight resource constraints of end devices.

³<https://optee.readthedocs.io/en/latest/building/devices/rpi3.html>

⁴<https://trustedfirmware-a.readthedocs.io/en/latest/>

⁵Inference Llama2 in pure C (<https://github.com/karpathy/llama2.c>), model configuration in (<https://github.com/karpathy/llama2.c/blob/master/train.py>)

TABLE IV
THE SECURE MEMORY SETTING OF MODELS USED FOR EVALUATION

Model	Parameter	Weight	Predictive Memory	Memory settings	Analysis latency
Mobilenet	5M	16.2 MB	36.2 MB	40 MB	294.1 ms
Darknet	6M	27.9 MB	51.9 MB	52 MB	273.4 ms
Resnet18	33M	43.9 MB	67.9 MB	68 MB	357.2 ms
Darknet19	41M	79.6 MB	103.7 MB	104 MB	406.1 ms
llama2	15M	67.1 MB	137.7 MB	140 MB	177.4 ms

SmartZone concerned with three key performance metrics: inference latency, parallel speedup, and energy consumption. Inference latency, which directly reflects the execution performance, is decomposed into stages, including the weight transfer process before inference, to assess the effectiveness of our optimization methods. Based on the latency, we also calculated the parallel speedup to reflect the parallel acceleration effect. Energy consumption measures the power and total energy consumption produced by performing on-device inference once, a commonly used evaluation metric on end devices. For comparison, we select three setups: (1) inferencing directly in the non-secure environment (marked as *REE*); (2) inferencing entirely in the native TEE environment on TrustZone with extended secure resources (marked as *TEE*); and (3) inferencing using SmartZone (marked as *SmartZone*). A detailed comparison between SmartZone and the related work has been described in Section II-A.

B. Secure Memory Setting

We evaluate SmartZone's secure memory setting on various models' inferences, as shown in Table IV. With pre-execution resource analysis, we calculate the required secure memory from each model's *cfg* files. For instance, MobileNet requires a maximum memory of 4 MB for a single layer, with an additional 16 MB earmarked for TEE OS, totaling 36.2 MB. Consequently, we set the secure memory size to 40 MB thus aligning with a 4 MB granularity. This loose allocation can ensure smooth model inference while maintaining security. As SmartZone releases the memory after each layer's inference, the overall secure memory requirement of DNN models is relatively low. Across the five models assessed, the secure memory allocations varied from 40 MB to 140 MB. Subsequent tests validated the correctness of the inference. Moreover, The latency of pre-execution resource analysis ranged from 177.4 ms to 406.1 ms, with llama2 showing the shortest latency due to fewer layers. Since the secure memory size estimation and configuration occur only once before model deployment, it does not affect inference performance.

C. Weight Transfer Latency

Weight transfer is a necessary stage before performing inference. This process is completed via the shared memory, where the REE side stores the encrypted weights, and the TEE side loads and decrypts them. SmartZone then builds the corresponding model in the secure world based on the obtained weights. Since the shared memory is dynamically allocated

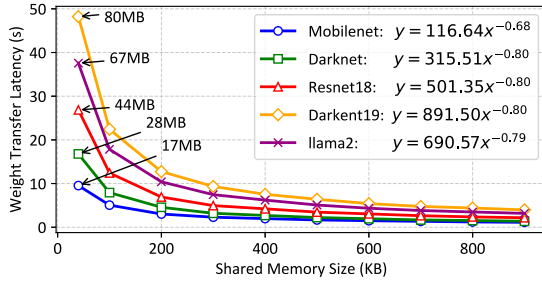


Fig. 7. Weight transfer latency for DNN and LLM inference on SmartZone across different shared memory configurations.

on the REE side, its size is not limited by secure resources. Considering that current storage devices are usually aligned to 4KB granularity, we set the size of shared memory in 4KB units and test the transfer latency for different sizes across different workloads. The detailed results are shown in Fig. 7.

The weight transfer for these workloads ranges from 17 MB to 80 MB. Generally, transfer latency increases with more weights being transferred. As the shared memory size increases, transfer latency gradually decreases, but the rate of reduction slows down. This is due to the multiple transfers based on shared memory size, each involving two time-consuming secure state switches. Small shared memory size requires hundreds of state switches, resulting in significant additional latency. Conversely, large shared memory reduces the number of switches, minimizing additional overhead, and the transfer latency stabilizes. In practical applications, we recommend setting the shared memory size to 600 KB or more, with a transfer latency ranging from 1.50s to 5.41s.

D. Performance on DNN Models

1) *Inference Latency*: Inference latency refers to the end-to-end execution latency to perform on-device inference once. We analyze end-to-end latency from two aspects: overall latency and latency changes due to SmartZone's design. Based on the inference process, we divide the overall latency into four parts: *im2col*, *gemm*, *normalize*, and *other*. The latency changes caused by design aim to evaluate the performance improvements brought by the trusted inference-oriented operator set, the proactive multi-threading parallel support, and the on-demand secure memory management of SmartZone. Each evaluation is conducted on four DNN workloads, across three cases of *REE*, *TEE*, and *SmartZone*. These tests are conducted with sufficient secure memory, as deploying the DNN model on TEE without memory expansion incurs significant performance overhead. Additionally, we test the latency of parallel acceleration on the REE side (marked as *REE-omp*), which uses OpenMP to parallelize the computation of the *gemm* stage, as existing frameworks do. For *SmartZone*, we further conduct the ablation test by removing each of the three optimization methods respectively, marked as *SmartZone-o1*, *SmartZone-o2*, and *SmartZone-o3*. The results of the inference latency are presented in Fig. 8.

Compared to *REE*, the execution performance of *TEE* is slower on Mobilenet, Darknet, Resnet18 and Darknet19 by 117%, 77%, 22%, and 28% respectively, mainly due to the inefficient *normalize* calculation. In the *gemm* stage, which does not involve secure state switching, the execution performance of *REE* and *TEE* is similar. With all three performance optimization methods, *SmartZone* shows competitive execution performance. Compared to *REE*, *SmartZone* achieves an average performance improvement of $2.50\times$. Compared to *TEE*, *SmartZone* yields performance improvements of $3.25\times$, $4.07\times$, $4.08\times$, and $4.26\times$ on the four models, respectively. On Resnet18 and Darknet19, which have large computational loads, *SmartZone* only brings an average latency overhead of 14.14% compared to the fully parallelized *REE-omp*.

Ablation tests show that the trusted inference-oriented operator set of SmartZone, especially the introduction of the mathematical operators *Tinylibm*, significantly enhances the computational efficiency of the *normalize* stage, providing an average performance improvement of 29.67% compared to *TEE*. The design of proactive multi-threading parallel support mainly aimed at the *gemm* stage to accelerate its computing efficiency. On Mobilenet, where the latency of *gemm* constitutes a relatively small proportion, parallel support brings a $1.86\times$ performance improvement. On Darknet, Resnet18, and Darknet19, it brings a $2.47\times$, $3.12\times$, and $3.13\times$ improvement respectively. The on-demand secure memory management of SmartZone performs resource analysis and static memory allocation before execution, introducing a negligible additional latency overhead, with an average overhead of 0.17%.

2) *Parallel Speedup*: Proactive multi-threading parallel support is crucial for SmartZone's strong performance when processing large-scale DNN models and serves as a key design that distinguishes SmartZone from existing solutions. In our implementation, we divide the inference process into the parallelizable *gemm* stage and the non-parallelizable *other* stage. We then test the execution latency and the speedup of *SmartZone* under various thread counts, comparing it with the mature OpenMP in REE. When enabling OpenMP, the default number of threads corresponds to the number of CPU cores. To fully evaluate the effect of the multi-threading parallelism, our tests range from single-thread to sixteen-thread execution. The results are shown in Fig. 9, where *ST* denotes single-thread execution, *MT2*, *MT3*, *MT4*, *MT8*, and *MT16* represent execution with 2, 3, 4, 8, 16 threads, respectively.

Since the REE thread scheduler allocates threads carrying computing tasks to different CPU cores for execution, the higher number of threads ideally results in lower execution latency and greater speedup. When the number of threads exceeds the number of cores, the speedup tends to be stable and might slightly decrease due to frequent thread switching affecting computing efficiency. For DNN models, the parallelizable *gemm* stage, which performs general matrix multiplication calculations, has strong parallelism. Consequently, in *REE-omp*, the parallel speedup of *gemm* stage closely approaches thread growth, particularly in Resnet18 and Darknet19, which have large computational loads. The overall speedup is influenced by the proportion of the *gemm* stage. We can approximate the

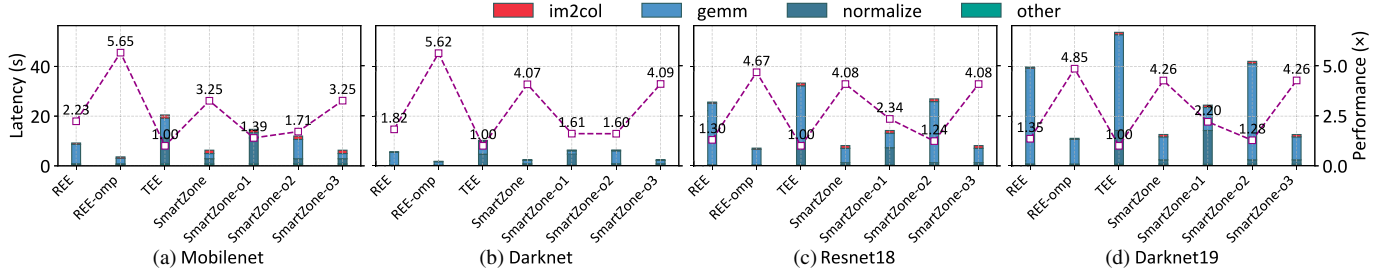


Fig. 8. End-to-end inference latency comparison among *REE* (native execution on REE), *REE-omp* (enabling OpenMP on REE), *TEE* (native execution on TEE), and *SmartZone* on DNN models. *SmartZone-o1/o2/o3* represent the performance of *SmartZone* when disabled trusted inference-oriented operator set, proactive multi-threading parallel support, and on-demand secure memory management, respectively, as part of ablation experiments.

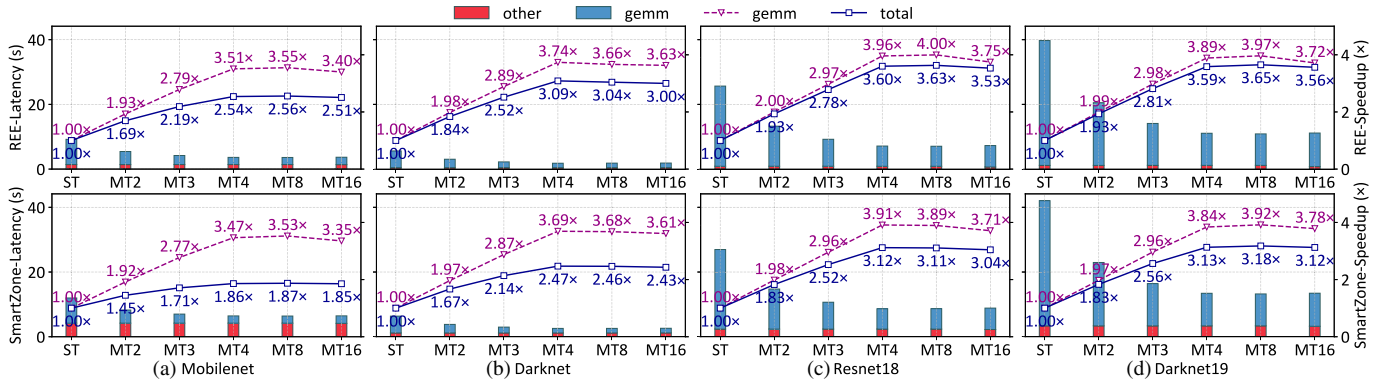


Fig. 9. Parallel speedup performance of *REE-omp* (enabling OpenMP on REE, top row) and *SmartZone* (bottom row) on a quad-core testbed across various thread counts. *ST* signifies a single-thread configuration, while *MT2*, *MT3*, *MT4*, *MT8*, and *MT16* refer to 2, 3, 4, 8, and 16 threads configurations, respectively.

degree of parallelism by comparing the execution latency of *gemm* stage to the overall execution latency in single-thread execution. The degree of parallelism for Mobilenet, Darknet, Resnet18, and Darknet19 ranges from 84.84% to 97.12%. Correspondingly, the overall speedup in *REE-omp* also increases, approaching the speedup of *gemm* stage. Finally, *REE-omp* achieves parallel speedups of 2.54 $\times$, 3.09 $\times$, 3.60 $\times$, and 3.59 $\times$ with four-thread execution under the four models. Among them, Mobilenet, which has the smallest computational workload, exhibits a relatively low overall speedup due to the high proportion of non-parallel stages. *SmartZone* follows a trend consistent with OpenMP through proactive multi-threading. The speedup of *gemm* stage is close to the ideal but slightly lower than *REE-omp* due to the cross-world thread collaboration in *SmartZone*, which involves secure state switching operations, introducing additional overhead. Additionally, the high execution latency of the *other* stage in *SmartZone* results in a lower overall parallel speedup compared to *REE-omp*, with the overall speedups for the four models at 1.86 $\times$, 2.47 $\times$, 3.12 $\times$, and 3.13 $\times$, respectively.

3) *Energy Consumption*: *SmartZone* is primarily designed for embedded devices, mobile devices, and other end devices with TrustZone support. Thus, power and energy consumption are considered crucial metrics. Fig. 10 illustrates the real-time average power during the on-device inference performed by *REE*, *TEE*, and *SmartZone* across four DNN models. When the Raspberry Pi 3B+ board runs without any workload (marked

as *noload*), its average power stabilizes at 2.12 W. The average power is influenced by the number of threads or the number of CPU cores utilized during inference. For *REE*, the average power during the single-thread execution is 2.95 W. The energy consumption, based on the execution latency, is 32.16 J, 19.39 J, 77.84 J, and 116.27 J on the four models. For *REE-omp*, the average power during four-thread execution is 4.28 W, 3.69 W, 4.30 W, and 4.45 W, respectively. As execution latency decreases, energy consumption is also significantly reduced, achieving a decrease of 51.00% compared to *REE*.

The average power of unoptimized *TEE* in single-thread execution mode is similar to *REE*, at 2.95 W. However, its high execution latency significantly increases energy consumption during inference. Compared to *REE*, *TEE* increases energy consumption by 98.76%, 62.97%, 25.43%, and 37.52% for the four models, respectively. Contrastly, *SmartZone* raises the average power to 4.29 W, 3.79 W, 4.09 W, and 4.40 W, respectively, through multi-threading parallelism. As it utilizes the same CPU resources, the average power of *SmartZone* is nearly the same as *REE-omp*. Compared to *TEE*, *SmartZone* significantly reduces energy consumption during inference, achieving an average reduction of 62.44% (ranging from 56.15% to 65.81%). For Darknet, Resnet18, and Darknet19, which require substantial computation, the energy consumption of *SmartZone* is very close to *REE-omp*. These results highlight that *SmartZone* significantly decreases energy consumption during inference while providing entire protection.

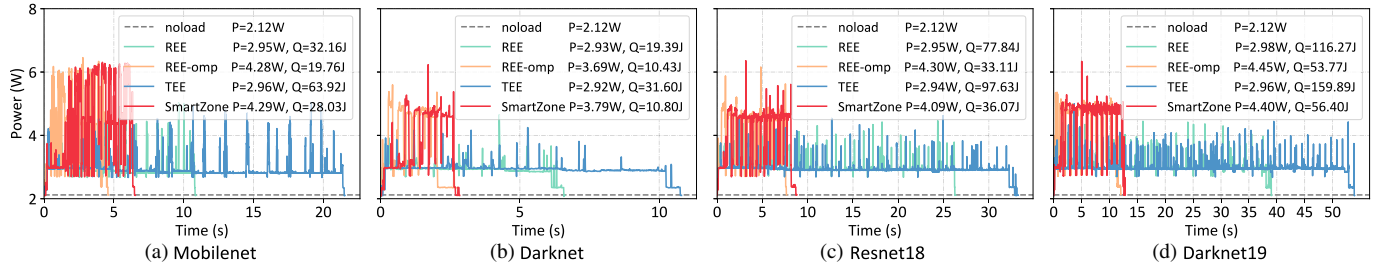


Fig. 10. Comparison of real-time power and total energy consumption among *REE* (native execution on REE), *REE-omp* (four threads parallelism via OpenMP on REE), *TEE* (native execution on TEE), and *SmartZone* (employing four threads) across four DNN workloads.

TABLE V
PERFORMANCE BREAKDOWN OF REE, TEE, AND SMARTZONE DURING EXECUTING LLAMA2

Stage			REE				TEE	SmartZone			
			ST	MT2	MT3	MT4	ST	ST	MT2	MT3	MT4
Preprocessing Latency (ms)	data transmission		/	/	/	/	4.44	4.35	4.45	4.24	4.54
	build transformer		1.92	1.46	1.36	1.56	2.04	2.02	1.37	1.48	1.55
	build tokenizer		129.03	128.16	122.92	134.52	131.15	131.69	127.33	118.83	140.54
	build sampler		0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.09	0.10
	encode		67.18	103.70	108.50	119.84	94.89	95.14	112.99	99.99	99.88
Decoding Latency for generating 256 tokens (s)	forward	matrix-vector multiplication	35.32	17.74	14.56	16.83	35.74	35.46	18.02	14.69	17.06
		multihead attention	2.91	1.53	1.14	1.22	2.95	2.93	1.55	1.15	1.24
		total time	38.59	19.61	16.05	18.41	38.60	38.76	19.91	16.18	18.67
	sample	decode	3.98	3.30	3.22	3.68	4.15	4.10	3.66	3.14	3.63
		total time	40.16	21.15	17.57	19.99	41.50	40.34	21.40	17.73	20.33
Postprocessing Latency(ms)	free sampler		0.16	0.14	0.18	0.18	0.24	0.19	0.35	0.23	0.19
	free tokenizer		14.47	14.54	16.18	18.02	23.41	18.54	17.44	24.34	18.48
	free transformer		19.55	17.69	19.81	21.28	30.57	23.65	21.23	29.93	22.34
Result Analysis	Seconds per token(s)		0.16	0.08	0.07	0.08	0.16	0.16	0.08	0.07	0.08
	Total latency(s)		40.56	21.51	17.90	20.41	41.73	40.79	21.84	18.02	20.72
	Throughput (token/s)		6.37	12.10	14.57	12.80	6.17	6.35	11.96	14.44	12.60
	Speedup (×)		/	1.90	2.29	2.01	/	/	1.88	2.28	1.98
	Power (W)		3.51	4.64	5.42	5.68	3.52	3.52	4.57	5.43	5.76
	Energy consumption (J)		61.32	46.72	46.97	53.98	64.19	63.03	46.92	47.15	53.82

E. Performance on LLM

The inference process of llama2, consistent with typical LLMs [43], can be divided into four stages: *preprocessing*, *prefill*, *decoding*, and *postprocessing*. The *prefill* stage primarily processes the user's prompt. As a story generation model, we utilize an empty prompt to showcase llama2's creativity. Thus, this stage is empty as a simple initialization. During *decoding* stage, llama2 generates tokens through *forward*, *sample*, and *decode* operations. The matrix-vector multiplication calculation and multi-head attention mechanism in the *forward* operation are prime for parallel acceleration. In our tests, we conduct 256 decoding cycles to generate 256 tokens. Table V provides the breakdown of latency for each stage along with performance results, highlighting the differences between *SmartZone* and *REE* in terms of memory operations and optimization methods.

1) *Inference Latency*: The results indicate that the average latency for the *preprocessing* stage is 130.33 ms in *REE*, compared to 137.74 ms in *TEE* and 135.69 ms in *SmartZone*, with additional latency due to weight transfer. In our large shared memory setting, this overhead is less than 4.54 ms. Inference performance differs between *REE* and *TEE* due to extra data transfer and slow memory operations during the *preprocessing*

and *postprocessing*. However, this discrepancy in total latency is relatively minor. Moreover, the memory operation latency on *TEE* shows less fluctuation than *REE*, likely due to its continuous dedicated secure memory, which is less impacted by fragmentation. The average time to generate each token shows that *TEE*'s latency is 8.04% higher than *REE*. For the *decoding* stage, which involves substantial computation, the overall execution latency of *REE*, *TEE*, and *SmartZone* is close. Most of the time during decoding is spent on matrix-vector multiplication, with overall latencies of 40.41 s, 21.43 s, 17.83 s, and 20.33 s for one to four threads in *REE*, and 40.42 s, 21.57 s, 17.89 s, and 20.49 s for *SmartZone*. The maximum inference speed for *REE* is 14.57 tokens/s, while *SmartZone* achieves 14.44 tokens/s. Overall, *SmartZone* reduces the inference latency by 56.74% compared to *TEE* and only increases it by 0.64% compared to *REE*.

2) *Parallel Speedup*: When enabling parallel acceleration, both *REE* and *SmartZone* deliver significant speedup. With OpenMP, *REE* achieves a speedup of 1.90×, 2.29×, and 2.01× in the two-thread to four-thread tests, respectively. Similarly, with proactive multi-threading parallel support, *SmartZone* achieves a speedup of 1.88×, 2.28×, and 1.98×, respectively. Due to the overhead of secure states switching in *SmartZone* for

thread management, there is a slight gap in the speedup ratio compared to *REE*, but both show the same acceleration trend. The highest speedup is observed with three threads. When using four threads, that is, all CPU cores, the acceleration decreases in both *REE* and *SmartZone*. This is because, during execution, the inference of each token requires thread allocation and scheduling, frequent thread management using all CPU cores affects the main thread controlling the workflow. In general, *SmartZone* achieves up to $2.34\times$ speedup over *TEE*, with a negligible performance gap of 0.09% compared to fully parallelized *REE*.

3) *Energy Consumption*: We further evaluate the energy consumption of llama2 to assess its on-device inference performance. For llama2 with extensive computations, the average power remains at a high peak during the *decoding* stage, depending on the number of CPU cores used. With one to four threads, the average power of *REE* is 3.51 W, 4.64 W, 5.42 W, and 5.68 W, while for *SmartZone* it is 3.52 W, 4.57 W, 5.43 W, and 5.76 W. *TEE* with parallel support has an average power of 3.52 W, similar to *REE* and *SmartZone* in single-thread mode. Due to the extended execution latency with single-thread mode, the energy consumption of *REE*, *TEE*, and *SmartZone* is relatively high at 64.19 J, 61.32 J, and 63.03 J. Multi-threading reduces the execution latency, leading to lower energy consumption, but also raises the average power. Thus the lowest energy consumption occurs with two threads in both *REE* and *SmartZone*. The energy consumption remains similar with three threads but increases with four threads due to diminished acceleration. Overall, *SmartZone* reduces energy consumption by 16.16% to 26.90% compared to *TEE* and is comparable to *REE*.

Experimental results on the small-scale LLM show the feasibility of secure and efficient on-device LLM inference. Such small-scale LLMs can be applied in two scenarios. The first involves domain-specific small-scale LLMs, which contain only knowledge of particular domains and require fewer parameters. For example, lightweight medical models can be integrated into smart fundus cameras to achieve privacy-sensitive medical image quality control. The second scenario is the distributed inference of large-scale LLMs. End devices can process a subset of computations within mixed-expert mode LLMs, or collaborate across multiple devices to perform distributed inference. Besides, *SmartZone*'s runtime support ensures that increases in model parameters do not affect the inference pattern and workflow, making it scalable and capable of supporting larger models.

VI. CONCLUSION

In this paper, we propose *SmartZone*, a runtime support for secure and efficient on-device inference on ARM TrustZone. We first construct a trusted inference-oriented operator set to provide underlying computing support for DNN models and LLMs, enabling the flexible construction of various models. *SmartZone* has achieved competitive performance with the proposed optimization technologies, proactive multi-threading parallel support, and on-demand secure memory management. Evaluations on real devices illustrate that *SmartZone*

can correctly execute typical DNN models and LLM inference with full-process protection while achieving up to $4.26\times$ speedup and reducing energy consumption by 65.81%.

REFERENCES

- [1] P. Guo, B. Hu, and W. Hu, "Mistify: Automating DNN model porting for on-device inference at the edge," in *Proc. 18th USENIX Symp. Netw. Syst. Des. Implementation (NSDI)*, 2021, pp. 705–719.
- [2] T. W. Rogers et al., "Evaluation of an AI system for the detection of diabetic retinopathy from images captured with a handheld portable fundus camera: The mailor AI study," *Nature Eye*, vol. 35, no. 2, pp. 632–638, 2021.
- [3] M. team, "MLC-LLM," 2023. Accessed: Sep. 17, 2024. [Online]. Available: <https://github.com/mlc-ai/mlc-llm>
- [4] F. Mo et al., "DarkneTZ: Towards model privacy at the edge using trusted execution environments," in *Proc. MobiSys*, 2020, pp. 161–174.
- [5] D. Cerdeira, J. Martins, N. Santos, and S. Pinto, "ReZone: Disarming TrustZone with TEE privilege reduction," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 2261–2279.
- [6] Z. Liu, Y. Lu, X. Xie, Y. Fang, Z. Jian, and T. Li, "Trusted-DNN: A TrustZone-based adaptive isolation strategy for deep neural networks," in *Proc. ACM TURC*, 2021, pp. 67–71.
- [7] Z. Jian et al., "TSC-VEE: A Trustzone-based smart contract virtual execution environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 6, pp. 1773–1788, Jun. 2023.
- [8] H. Yan et al., "LDR: Secure and efficient linux driver runtime for embedded TEE systems," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2024.
- [9] Z. Li, W. Li, Y. Xia, and B. Zang, "Teep: Supporting secure parallel processing in arm trustzone," in *Proc. IEEE 26th Int. Conf. Parallel Distrib. Syst.*, Piscataway, NJ, USA: IEEE, 2020, pp. 544–553.
- [10] M. S. Islam, M. Zamani, C. H. Kim, L. Khan, and K. W. Hamlen, "Confidential execution of deep learning inference at the untrusted edge with arm trustzone," in *Proc. 13th ACM Conf. Data Appl. Secur. Privacy*, 2023, pp. 153–164.
- [11] X. Xie et al., "Memory-efficient and secure DNN inference on trustzone-enabled consumer IoT devices," in *Proc. IEEE INFOCOM—IEEE Conf. Comput. Commun.*, 2024, pp. 2009–2018.
- [12] Y. Wang, K. Deng, F. Meng, Z. Chen, M. Yin, and R. Yang, "Branchy-TEE: Deep learning security inference acceleration using trusted execution environment," in *Proc. 35th Int. Conf. Softw. Eng. Knowl. Eng.*, 2023, pp. 499–504.
- [13] Y. Li et al., "Efficient and secure deep learning inference in trusted processor enabled edge clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4311–4325, Dec. 2022.
- [14] Y. Xiang, Y. Wang, H. Choi, M. Karimi, and H. Kim, "AegisDNN: Dependable and timely execution of DNN tasks with SGX," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Piscataway, NJ, USA: IEEE, 2021, pp. 68–81.
- [15] F. Tramer and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [16] S. A. Osia et al., "A hybrid deep learning architecture for privacy-preserving mobile analytics," *IEEE Internet Things J.*, vol. 7, no. 5, pp. 4505–4518, May 2020.
- [17] A. Gangal, M. Ye, and S. Wei, "HybridTEE: Secure mobile DNN execution using hybrid trusted execution environment," in *Proc. Asian Hardware Oriented Secur. Trust Symp.*, Piscataway, NJ, USA: IEEE, 2020, pp. 1–6.
- [18] K. G. Narra, Z. Lin, Y. Wang, K. Balasubramanian, and M. Annavaram, "Origami inference: Private inference using hardware enclaves," in *Proc. IEEE 14th Int. Conf. Cloud Comput.*, 2021, pp. 78–84.
- [19] H. Hashemi, Y. Wang, and M. Annavaram, "DarKnight: An accelerated framework for privacy and integrity preserving deep learning using trusted hardware," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitect. (MICRO)*, 2021, pp. 212–224.
- [20] Q. Li, J. Ren, X. Pan, Y. Zhou, and Y. Zhang, "ENIGMA: Low-latency and privacy-preserving edge inference on heterogeneous neural network accelerators," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Piscataway, NJ, USA: IEEE, 2022, pp. 458–469.
- [21] T. Shen et al., "SOTER: Guarding black-box inference for general neural networks at the edge," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2022, pp. 723–738.

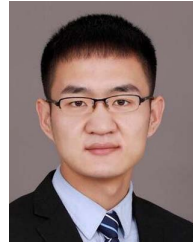
- [22] Z. Sun, R. Sun, C. Liu, A. R. Chowdhury, L. Lu, and S. Jha, "ShadowNet: A secure and efficient on-device model inference system for convolutional neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE, 2023, pp. 1596–1612.
- [23] B. Hu et al., "Secure and efficient mobile DNN using trusted execution environments," in *Proc. ACM Asia Conf. Comput. Commun. Secur. (ASIA CCS)*, 2023, pp. 274–285.
- [24] Z. Zhang et al., "No privacy left outside: On the (in-) security of tee-shielded DNN partition for on-device ml," in *Proc. S&P*, 2024, pp. 3327–3345.
- [25] Z. Liu, Y. Luo, S. Duan, T. Zhou, and X. Xu, "MirrorNet: A tee-friendly framework for secure on-device DNN inference," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des.*, 2023, pp. 1–9.
- [26] Z. Liu, T. Zhou, Y. Luo, and X. Xu, "TBNNet: A neural architectural defense framework facilitating DNN model protection in trusted execution environments," in *Proc. DAC*, 2024, pp. 1–6.
- [27] R. Liu, L. Garcia, Z. Liu, B. Ou, and M. Srivastava, "SecDeep: Secure and performant on-device deep learning inference framework for mobile and IoT devices," in *Proc. Int. Conf. Internet-Things Des. Implementation*, 2021, pp. 67–79.
- [28] W. Hua, M. Umar, Z. Zhang, and G. E. Suh, "GuardDNN: Secure accelerator architecture for privacy-preserving deep learning," in *Proc. 59th ACM/IEEE Des. Automat. Conf. (DAC)*, 2022, pp. 349–354.
- [29] J. Jiang et al., "CRONUS: Fault-isolated, secure and high-performance heterogeneous computing for trusted execution environment," in *Proc. 55th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, Piscataway, NJ, USA: IEEE, 2022, pp. 124–143.
- [30] Y. Deng et al., "Strongbox: A GPU TEE on Arm endpoints," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2022, pp. 769–783.
- [31] S. Siby, S. Abdollahi, M. Maheri, M. Kogias, and H. Haddadi, "GuardanTEE: Towards attestable and private ML with CCA," in *Proc. 4th Workshop Mach. Learn. Syst.*, 2024, pp. 1–9.
- [32] D. Li, Z. Mi, Y. Xia, B. Zang, H. Chen, and H. Guan, "TwinVisor: Hardware-isolated confidential virtual machines for ARM," in *Proc. ACM SIGOPS 28th Symp. Operat. Syst. Principles (SOSP)*, 2021, pp. 638–654.
- [33] S.-K. Han and J. Jang, "MyTEE: Own the trusted execution environment on embedded devices," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2023.
- [34] O. Ohrimenko et al., "Oblivious Multi-Party machine learning on trusted processors," in *Proc. USENIX Secur.*, 2016, pp. 619–636.
- [35] X. Jiang, M. Kim, K. Lauter, and Y. Song, "Secure outsourced matrix computation and application to neural networks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 1209–1222.
- [36] R. Tang, M. Du, N. Liu, F. Yang, and X. Hu, "An embarrassingly simple approach for Trojan attack in deep neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining (SIGKDD)*, 2020, pp. 218–228.
- [37] B. Biggio and F. Roli, "Wild patterns: Ten years after the rise of adversarial machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2018, pp. 2154–2156.
- [38] Y. Liu, L. Wei, B. Luo, and Q. Xu, "Fault injection attack on deep neural network," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, Piscataway, NJ, USA: IEEE, 2017, pp. 131–138.
- [39] R. S. Chakraborty, I. Saha, A. Palchoudhuri, and G. K. Naik, "Hardware Trojan insertion by direct modification of FPGA configuration bitstream," *IEEE Des. Test*, vol. 30, no. 2, pp. 45–54, 2013.
- [40] X. Lou, T. Zhang, J. Jiang, and Y. Zhang, "A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptography," *ACM Comput. Surveys (CSUR)*, vol. 54, no. 6, pp. 1–37, 2021.
- [41] K. Ryan, "Hardware-backed heist: Extracting ECDSA keys from qualcomm's trustzone," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 181–194.
- [42] R. Eldan and Y. Li, "Tinystories: How small can language models be and still speak coherent English?" 2023, *arXiv:2305.07759*.
- [43] W. Kwon et al., "Efficient memory management for large language model serving with pagedattention," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 611–626.



Zhaolong Jian received the B.Eng. degree in Internet of Things engineering in 2020 from Nankai University, where he is currently working toward the Ph.D. degree with the College of Computer Science. His research interests include trusted execution environment, system security, and blockchain virtual machine.



Xu Liu received the M.Eng. degree from Tiangong University, Tianjin, China, in 2024. He is currently working toward the Ph.D. degree with the College of Computer Science, Nankai University. His research interests include edge computing, trusted execution environment, and the Internet of Things.



Qiankun Dong received the M.A.Sc. degree in computer science from Nankai University, Tianjin, China, in 2016. Currently, he is a Professional and Technical Personnel with the College of Computer Science, Nankai University. His research interests include heterogeneous computing, Internet of Things security, and artificial intelligence. He is a member of CCF.



Longkai Cheng received the B.Eng. degree from China University of Petroleum (East China), Qingdao, China, in 2023. He is currently working toward the master's degree with the College of Computer Science, Nankai University. His research interests include model compression and edge computing.



Xueshuo Xie received the Ph.D. degree in computer science from Nankai University, in 2021, and the B.S. and M.A.Sc. degrees from Shandong University, Jinan, China, in 2011 and 2014, respectively. Currently, he is an Associate Researcher with Haihe Lab of ITAI. His research interests include Internet of Things security, data-driven anomaly detection, and blockchain.



Tao Li (Member, IEEE) received the Ph.D. degree in computer science from Nankai University, China, in 2007. Currently, he is a Professor with the College of Computer Science, Nankai University, and a Deputy Head with Haihe Lab of ITAI. His research interests include heterogeneous computing, machine learning, and intelligent Internet of Things. He is a member of ACM and a distinguished member of CCF.